

LEARNING RESOURCE GUIDE

ISC2 CCSP: Certified Cloud Security Professional (Aug 2026) | Professional Participant Workbook

Learning Objectives

By the end of this lecture and its accompanying guide, you will be able to:

- By the end of this lecture, you will be able to identify the distinct threat profiles associated with each major cloud storage type: object, volume, ephemeral, and raw storage.
- By the end of this lecture, you will be able to explain how the Capital One 2019 breach demonstrates the chaining of storage threats from compute misconfiguration to object storage data exposure.
- By the end of this lecture, you will be able to describe the risks of orphaned volumes, unencrypted volumes, exposed snapshots, and data remanence in cloud environments.
- By the end of this lecture, you will be able to articulate why encryption alone is insufficient as a defense against all cloud storage threat categories.
- By the end of this lecture, you will be able to design a layered defense strategy addressing the specific threats of each storage type, including access policy, monitoring, encryption, and lifecycle management.

Introduction

In 2019, a security researcher identified thousands of publicly accessible Amazon S3 buckets containing sensitive organizational data. The buckets had not been hacked. They had not been subjected to sophisticated attack techniques. They were simply configured to allow public read access — either intentionally for a temporary purpose that was never revisited, or accidentally by someone who did not fully understand the permission model. Terabytes of data, including customer records, financial documents, and source code, was available to anyone who knew where to look. This was not an isolated incident; it established a pattern that the cloud security community recognizes as one of the defining threat categories in cloud data security.

Misconfiguration is the single most pervasive threat to cloud storage, but it is far from the only one. Each storage type — object, volume, ephemeral, and raw — has a distinct threat profile shaped by how it works and how it is accessed. The Capital One breach of 2019, which resulted in the exposure of over one hundred million customer records and an eighty-million-dollar regulatory fine, illustrates how threats chain across storage types: a compute misconfiguration provided the initial foothold, overly permissive identity credentials made the S3 bucket accessible, and a principle-of-least-privilege failure transformed a bad situation into a catastrophic one.

This guide maps the specific threat landscape for each storage type encountered in the CCSP curriculum. Understanding these threats with the precision required by the examination — and by professional practice — means moving beyond the generic concept of 'data breach' and into the concrete mechanisms: which misconfiguration, which credential, which neglected backup, which orphaned resource. The defense is always proportional to how well the threat is understood.

Core Concepts Explained

Object Storage Misconfiguration: Bucket Policy and ACL Threats

Definition: Misconfiguration of object storage access controls — specifically bucket policies and access control lists that grant public or excessively broad read/write access — is the most pervasive and consequential threat in cloud data security. It does not require an attack; it is a configuration state that exposes data without any adversarial action.

Business relevance: Public bucket misconfiguration has resulted in some of the largest data exposures in cloud history, affecting healthcare providers, government agencies, financial institutions, and technology companies. The business impact includes regulatory fines, breach notification costs, reputational damage, and loss of customer trust. Because the exposure is not an intrusion, it often goes undetected for extended periods.

Cloud Security Practitioner relevance: As a Cloud Security Practitioner, preventing and detecting misconfiguration is a core operational responsibility. This means not only applying correct bucket policies initially but establishing continuous detection: automated rules that alert within minutes of any bucket being set to public access, and periodic audits that review all externally accessible storage resources.

Real-world example: A marketing team at a retail company uploads creative assets and customer survey results to an Amazon S3 bucket during a campaign. To allow a third-party design agency quick access during the campaign, a developer sets the bucket to public-read for two weeks. The campaign ends, the agency relationship concludes, and the developer moves to another project. Eighteen months later, a bug bounty researcher discovers the bucket is still public. It contains survey responses including customer names, email addresses, and purchase preference data for 340,000 customers. The company did not have automated detection for public bucket states. The researcher reported it responsibly, but the company had no idea how long the data had been exposed. The remediation required disabling public access, notifying affected customers, and engaging external legal counsel to assess breach notification obligations in three jurisdictions. The practice that prevents this is enabling Block Public Access at the organization level and deploying an AWS Config rule — or equivalent CSPM policy — that fires an alert and triggers automatic remediation within five minutes of any public access grant on any bucket.

Risks or limitations: Risks include stale public configurations from past use cases, developer shortcuts during testing that are not reverted, and miscommunication between teams about who owns the bucket post-project. Best practice: Block Public Access at the account and organization level, audit all bucket policies and ACLs quarterly using cloud-native tools (S3 Access Analyzer, AWS Config), and require documented approval for any intentional external sharing with a defined expiry date.

Insecure API Access and Credential Compromise in Object Storage

Definition: Object storage is accessed via HTTP-based APIs authenticated by credentials — IAM access keys, IAM roles, presigned URLs, or temporary STS tokens. Compromise, misuse, or over-permission of any of these credentials gives the holder full API access to whatever the credential permits, regardless of network controls.

Business relevance: The Capital One 2019 breach is the definitive reference case. An attacker exploited a misconfigured WAF on a cloud compute instance to execute a server-side request forgery (SSRF) attack, queried the EC2 instance metadata service, retrieved temporary IAM credentials, and used those credentials to download over one hundred million customer records from an S3 bucket. The fine from the OCC (Office of the Comptroller of the Currency) was eighty million dollars. The credentials had broader S3 permissions than the application required — a principle-of-least-privilege failure that converted a recoverable compute misconfiguration into a catastrophic data breach.

Cloud Security Practitioner relevance: A Cloud Security Practitioner must treat every credential that grants S3 API access as a potential attack vector. The controls are: scoped IAM roles that grant only the permissions the application actually uses, no long-lived access keys in application code, monitoring of API activity for anomalous patterns, and active protection of the instance metadata service (IMDSv2 enforcement on EC2 to prevent SSRF-based credential theft).

Real-world example: A cloud security team conducting a threat model exercise at a fintech company identifies that their transaction processing service runs on EC2 with an IAM role that has S3 read access to both the transaction log bucket and the customer PII archive bucket. The application only ever reads from the transaction log bucket; the PII archive access was added during initial role setup 'in case it was needed later' and was never scoped down. The team recommends splitting the role: the transaction service gets a tightly scoped role with read access to only the transaction log prefix, while the PII archive role is restricted to a separate compliance function with additional approval controls and access logging alerts. They also enforce IMDSv2 on all EC2 instances in the account, requiring a session-oriented token to retrieve instance metadata credentials, closing the SSRF-to-IMDS credential theft path that was the Capital One breach's initial escalation mechanism.

Risks or limitations: Key risks include SSRF vulnerabilities enabling instance metadata credential theft (closed by IMDSv2), long-lived access keys committed to code (remediated by role-based auth and secrets scanning), and IAM roles with excessive permissions (addressed by least privilege and periodic IAM access advisor reviews). Best practice: enforce IMDSv2 across all compute, use IAM Access Analyzer to identify external access and over-permissive policies, and implement automated alerts for API calls that exceed the baseline pattern for any given identity.

Volume Storage Threats: Unencrypted Volumes, Orphaned Resources, and Snapshot Exposure

Definition: Volume storage threats differ structurally from object storage threats. The primary vectors are: unencrypted volumes accessible at block level when the attached instance is compromised; orphaned detached volumes retaining sensitive data after workload decommissioning; and snapshot sharing misconfigurations that expose complete block-level backups to unintended accounts.

Business relevance: An unencrypted EBS volume attached to a compromised EC2 instance gives an attacker direct, unmediated access to every byte on the volume. Orphaned volumes in dynamic cloud environments — where instances are frequently provisioned and terminated — accumulate quietly and may contain sensitive production data from months or years prior. Publicly shared snapshots have been discovered containing complete production database backups, configuration files with embedded credentials, and sensitive customer data.

Cloud Security Practitioner relevance: A Cloud Security Practitioner must maintain a clear operational picture of the encryption status of all volumes and the sharing configuration of all snapshots. This requires automated tooling, not manual review: detective controls that fire on unencrypted volume creation, CSPM policies that alert on snapshot sharing changes, and a defined lifecycle for detached volumes that sets a disposition SLA.

Real-world example: During a cloud cost optimization review at a logistics company, an engineer examining unused resources flags 47 EBS volumes in a detached state. The volumes had been attached to EC2 instances that were terminated as part of a container migration six months earlier. A security review reveals that 12 of the 47 volumes originated from database instances that processed order and payment data. None of the volumes are encrypted — the database encryption strategy relied on the database-layer TDE (Transparent Data Encryption) without underlying disk encryption. The security team mounts three sample volumes on an isolated analysis instance and confirms the presence of database data files containing transaction records. Remediation involves creating encrypted snapshots of all sensitive volumes, deleting the originals, and confirming the

encrypted copies as archive records before scheduling them for deletion after the retention period. The team implements an SCP that denies unencrypted EBS volume creation going forward and establishes a monthly orphaned-resource report as a standing operational control.

Risks or limitations: Unencrypted volumes require disk-level protection separate from application-layer encryption. Orphaned volumes require automated detection and a disposal SLA. Snapshot sharing requires separate configuration review. Best practice: enforce default EBS encryption at the region level, detect orphaned volumes with automated weekly reporting, and treat any snapshot sharing change as a security event requiring review.

Ephemeral Storage: Data Remanence and the Transience Fallacy

Definition: Ephemeral storage is instance-local storage that does not persist after instance termination. The two threats specific to it are data remanence — residual data on storage media after a workload has ended — and the transience fallacy: the assumption that data on ephemeral storage requires fewer security controls because it will soon be deleted.

Business relevance: Cloud providers generally implement protections to wipe storage before reallocation to another tenant, but data on ephemeral storage remains fully accessible during the instance's active lifetime, which may be hours or days. Sensitive credentials, decrypted records, session tokens, and plaintext copies of otherwise encrypted data may all be present on ephemeral storage during an active workload, making it a live target for any attacker who compromises the instance.

Cloud Security Practitioner relevance: A Cloud Security Practitioner must challenge any architectural decision that places sensitive data on ephemeral storage with reduced controls on the basis that 'it will be gone soon.' The data is real and accessible during the processing window. Controls appropriate for persistent storage — access controls, in-memory data minimization, secure deletion of temp files before instance termination — apply equally to ephemeral storage.

Real-world example: A machine learning team at a pharmaceutical company processes de-identified clinical trial data on EC2 instances using ephemeral NVMe storage for intermediate computation files. The team's security questionnaire indicates the data is 'low sensitivity' because it is de-identified. However, a security review identifies that the intermediate computation files include a mapping table between de-identified patient IDs and original patient identifiers — a file generated automatically by the preprocessing pipeline and never flagged for special handling. The mapping table means the ephemeral storage holds, for the duration of each computation run (typically 3–6 hours), the ability to re-identify hundreds of thousands of clinical subjects. The security practitioner requires the preprocessing pipeline to be refactored to keep the mapping table in a separate encrypted persistent volume with restricted access, never written to ephemeral storage. The incident illustrates how 'transient' does not mean 'unexposed.'

Risks or limitations: Data remanence risk is generally mitigated by cloud provider controls at termination, but data in active use on ephemeral storage is exposed for the entire instance lifetime. The transience fallacy leads to absent access controls, unmonitored process execution, and sensitive temp files. Best practice: apply the same data classification and access controls to ephemeral storage that you would to persistent storage for any sensitive workload, and implement secure deletion of sensitive temporary files before instance termination.

Layered Defense Requirement: Why Encryption Alone is Insufficient

Definition: A common misconception in cloud storage security is that encrypting data is a sufficient defense. Encryption protects data from unauthorized access by parties who do not hold decryption keys — but it does not prevent authorized credential holders with excessive permissions from exfiltrating data, and it does not prevent configuration errors that expose storage to the public internet.

Business relevance: The Capital One breach illustrates this precisely: the attacker used legitimate, albeit stolen, credentials to access S3 data. If the bucket's data had been encrypted with server-side encryption managed by AWS (the most common configuration), the attacker's API calls would have returned the data in plaintext — because AWS would have decrypted it on behalf of the authenticated API caller. Encryption protected the storage medium, not the API.

Cloud Security Practitioner relevance: A Cloud Security Practitioner must design and articulate a layered defense strategy that addresses the full threat spectrum for each storage type. For object storage: encryption at rest, well-scoped IAM policies, no public access, active API monitoring, and lifecycle management of credentials. For volume storage: encryption of volumes and snapshots, instance hardening, least-privilege OS access, snapshot permission auditing, and orphaned resource lifecycle policies.

Real-world example: A CISO at a regional bank presents a board-level risk report claiming that all cloud storage is 'encrypted and therefore protected.' The bank's Cloud Security Practitioner prepares a memo demonstrating four failure modes that encryption alone does not address: (1) A developer's IAM key with broad S3 read access is committed to a public GitHub repository — encryption does not prevent the key-holder from reading plaintext via API; (2) An S3 bucket is set to public read — encryption is irrelevant because the provider decrypts on behalf of any requester; (3) A production EBS snapshot is shared with an external account — the snapshot may be encrypted, but the external account can mount it if they have access to the key; (4) An EC2 instance with excessive IAM permissions is compromised via a web application vulnerability — the attacker uses the instance's IAM role to call S3 GetObject and retrieves decrypted data. The memo recommends a layered control framework and prompts a board-level discussion on the difference between encryption as a compliance checkbox and encryption as one component of a comprehensive defense.

Risks or limitations: Relying solely on encryption creates a false sense of security, leaving organizations vulnerable to credential compromise, IAM misconfiguration, public access errors, and insider threats. Best practice: treat encryption as a necessary but insufficient control and pair it with IAM least privilege, access monitoring, public access prevention, credential hygiene, and snapshot governance.

Practical Applications

The following scenarios show how these threat categories manifest in operational practice and how Cloud Security Practitioners detect, respond to, and prevent them.

Threat-Modeling a Multi-Tier Cloud Storage Architecture

A Cloud Security Practitioner is engaged during the design of a new e-commerce platform that will use Amazon S3 for product images and customer order documents, EBS volumes for the PostgreSQL order database, and ephemeral EC2 instance storage for session caching. The practitioner conducts a threat-modeling session using the STRIDE model (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege). For the S3 layer, the highest-risk findings are information disclosure via misconfigured bucket policy and credential theft via SSRF against the EC2 application tier. The practitioner recommends IMDSv2 enforcement, IAM roles scoped to only the S3 prefixes each application component needs, Block Public Access at the account level, and CloudTrail data event logging feeding a SIEM alert for bulk GetObject API calls. For the EBS layer, the threats are unencrypted volumes attached to a compromised instance and unprotected snapshots during backup operations. The practitioner requires default EBS encryption using a customer-managed KMS key and an automated snapshot permission audit after every backup job. For the ephemeral session cache, the practitioner requires that no customer PII be written to the ephemeral layer; session tokens must reference data

in the database rather than cache the data itself. The threat model is documented as a design decision record and reviewed at each sprint milestone.

Investigating an S3 Exfiltration Alert

A SIEM alert fires at 2:47 AM on a Tuesday: an IAM role associated with a background processing Lambda function has made 14,000 GetObject calls against an S3 bucket in a 12-minute window. The normal pattern for that role is approximately 50 GetObject calls per hour during business hours. The Cloud Security Practitioner on call opens an investigation. The CloudTrail data event logs confirm the calls originated from the Lambda function's execution environment, all within a specific Lambda invocation ID. Cross-referencing with the Lambda function's deployment history shows that the function was updated six hours earlier with a code change submitted by an external contractor. The code change added an API call that exports all S3 objects matching a prefix to an external endpoint, disguised as a 'data sync' function. The practitioner immediately revokes the Lambda function's execution role, blocks the external endpoint at the network policy layer, initiates a review of all contractor code submissions since the contractor's engagement began, and notifies the incident response team. The SIEM alert was possible because CloudTrail data events were enabled and a baseline anomaly detection rule had been configured for that specific IAM role. Without the logging and the alert, the exfiltration would not have been detected until a scheduled quarterly access review.

Establishing a Volume Lifecycle Security Program

A Cloud Security Practitioner joins an organization that has been running cloud workloads for four years without a formalized volume lifecycle program. The practitioner begins with a discovery phase: a script queries all AWS regions in all organization accounts for EBS volumes in a 'detached' state. The output reveals 312 detached volumes across 14 accounts. The practitioner works with account owners to classify each volume: originating workload type, date of last attachment, presence of encryption, estimated data sensitivity based on the source workload's data classification. Eighteen volumes from database instances handling customer payment data are prioritized for immediate secure disposal. Volumes from low-sensitivity development environments are scheduled for deletion after a 30-day hold. The practitioner implements three ongoing controls: (1) An AWS Config rule alerts on any EBS volume that remains in a detached state for more than 14 days without a documented hold; (2) An SCP denies creation of new EBS volumes without encryption; (3) A monthly orphaned-resource report is generated automatically and reviewed by each account owner. The program is documented, with the volume lifecycle policy integrated into the organization's cloud governance framework.

Best Practices

- **Enable and enforce IMDSv2 (Instance Metadata Service v2) on all EC2 instances to close the SSRF-to-credential-theft attack path demonstrated in the Capital One breach.** IMDSv2 requires a session-oriented token to retrieve instance metadata credentials, unlike IMDSv1 which responds to any HTTP GET from the instance network. An SSRF vulnerability can make requests on behalf of the server, but with IMDSv2, the attacker must also execute a PUT request to obtain the session token — a two-step process that most SSRF payloads cannot complete. Enforce IMDSv2 via AWS SCP or EC2 instance metadata options on all launch templates, and verify compliance using AWS Config's ec2-imdsv2-check managed rule.
- **Implement automated detective controls that alert within minutes of any public access grant on object storage, rather than relying on periodic manual audits.** Manual quarterly audits of bucket policies are insufficient: a bucket misconfigured today may expose data for months before the next audit cycle. AWS Config managed rules (s3-bucket-public-read-prohibited, s3-bucket-public-write-prohibited) combined with automatic remediation Lambda functions can detect and revert public access grants within minutes. Pair

these with Amazon GuardDuty's S3 protection to detect anomalous access patterns in real time. Set a documented SLA: any public bucket misconfiguration alert must be acknowledged and remediated within one business hour.

- **Apply the principle of least privilege to every IAM role that accesses object storage, scoping permissions to specific S3 prefixes and actions, not entire buckets or accounts.** The Capital One breach's blast radius was amplified by an IAM role that had broader S3 read permissions than the application required. Use IAM policy conditions to scope roles to specific S3 prefixes (s3:prefix), limit GetObject to the exact paths needed, and remove ListBucket where enumeration is not required. Use AWS IAM Access Analyzer and IAM Access Advisor to identify and remove permissions that have not been used in the past 90 days. Review IAM role permissions at every application deployment and include a least-privilege review as a mandatory step in the change management process.
- **Enforce default encryption for all EBS volumes at the region level and audit all existing unencrypted volumes on a defined remediation schedule.** AWS allows enabling EBS encryption by default at the region level, so any newly created volume is automatically encrypted with a KMS key without requiring the requestor to specify it. Enable this setting in every region in every account via an SCP. For existing unencrypted volumes: create an encrypted snapshot, restore a new encrypted volume from the snapshot, detach the old volume, attach the new one, verify, and delete the original. Track remediation progress with a monthly report until all volumes are encrypted.
- **Establish a formal snapshot lifecycle policy that governs creation, retention, sharing permissions, and expiry of all volume snapshots.** Snapshot security failures — publicly shared backups, shared snapshots to inactive accounts, snapshots retained beyond their compliance period — stem from the absence of formalized governance rather than complex technical attack. Define a snapshot lifecycle policy: retention periods by data classification, explicit prohibition on public sharing, automated deletion on schedule, and quarterly review of snapshot sharing configurations. Implement AWS Data Lifecycle Manager or equivalent to automate snapshot creation and deletion on schedule, reducing reliance on manual operational steps that introduce inconsistency.

Common Mistakes and Misconceptions

Misconception: Assuming that server-side encryption of S3 objects prevents unauthorized access by someone who has obtained valid IAM credentials for the bucket.

Correct approach: Server-side encryption protects the data at rest on the storage medium. When the cloud provider decrypts data on behalf of an authenticated API caller, the encryption offers no protection against a credential holder — whether legitimate or a thief — who has API access. The correct defense is least-privilege IAM policy design, anomaly-based API monitoring, and credential hygiene.

Misconception: Treating ephemeral instance storage as inherently low-risk because the data disappears when the instance terminates.

Correct approach: Data on ephemeral storage is fully accessible during the entire instance lifetime. An instance running for hours or days processing sensitive data provides a real attack window. Apply the same data classification and access controls to ephemeral storage that you would to persistent storage for any sensitive workload.

Misconception: Relying on the encryption state of a volume to confirm that derived snapshots are equally protected from unauthorized sharing.

Correct approach: Snapshot sharing permissions are independent of volume encryption. An encrypted snapshot can be made publicly accessible through a misconfigured sharing policy. Treat snapshot permission auditing as a separate, mandatory control distinct from volume encryption review.

Misconception: Not rotating or revoking IAM credentials after an EC2 instance is terminated, assuming the credentials are invalidated automatically.

Correct approach: IAM access keys attached to IAM users are not invalidated when an EC2 instance is terminated. Only short-lived STS credentials issued to IAM roles expire automatically. Any static access key that was used by a terminated instance remains valid until explicitly rotated or revoked. Audit and revoke or rotate all access keys associated with decommissioned workloads as part of the instance termination process.

table: Cloud Storage Threat Reference: Type, Threat, and Layered Control

Use this reference table during security reviews and threat-modeling sessions to ensure coverage of the distinct threat categories for each storage type.

Storage Type	Primary Threats	Key Detective Controls	Key Preventive Controls
Object (S3, Blob, GCS)	Misconfigured public access; credential compromise (SSRF); insider exfiltration	CloudTrail data events; GuardDuty S3 protection; SIEM anomaly alerts	Block Public Access; least-privilege IAM; IMDSv2; S3 Access Analyzer
Volume (EBS, Managed Disk)	Unencrypted volumes; snapshot exposure; orphaned volumes post-decommission	Config rules: unencrypted volume alert; snapshot sharing change alert	Default region-level encryption; snapshot lifecycle policy; SCP deny unencrypted volumes
Ephemeral (instance local)	Data remanence during active use; transience fallacy (reduced controls)	Instance behavioral monitoring; data classification review of workloads	Apply same classification controls as persistent storage; avoid sensitive data writes to ephemeral store
Raw storage	Absence of file-system access controls; residual data from prior workloads	Application-layer audit logging; penetration testing of applications using raw storage	Application must implement all access validation; sanitize/zero-fill before allocation

How to use this: Review this table at the start of any cloud architecture review. Confirm that all four storage types in scope have identified threats mapped to specific implemented controls — not generic security language.

Knowledge Check

Test your understanding before moving on. Answers are shown below each question.

Q1. The Capital One 2019 breach began with a misconfigured web application firewall and resulted in the exposure of over 100 million customer records from Amazon S3. Which security principle failure most directly amplified the breach's impact?

- A) A. The S3 buckets were not encrypted at rest, allowing direct plaintext access to the data.
- B) B. The IAM role used by the application had broader S3 permissions than the application required, violating the principle of least privilege.
- C) C. The WAF was deployed in a public subnet rather than a private subnet, exposing the management interface.

D) D. The instance was not running in a VPC, allowing direct internet access to the metadata service.

Answer: B. The IAM role used by the application had broader S3 permissions than the application required, violating the principle of least privilege.

Q2. A cloud security team discovers 60 EBS volumes in a detached state across the organization's accounts. Several originated from database instances. What is the most appropriate immediate action for the security team?

- A) A. Delete all detached volumes immediately to eliminate the risk of unauthorized access.
- B) B. Classify the volumes by originating workload sensitivity, prioritize those from sensitive workloads for encrypted archiving or secure disposal, and implement automated alerts for future orphaned volumes.
- C) C. Re-attach all volumes to running instances to confirm they are encrypted before taking further action.
- D) D. The volumes pose no risk because they are detached and not accessible through any running compute instance.

Answer: B. Classify the volumes by originating workload sensitivity, prioritize those from sensitive workloads for encrypted archiving or secure disposal, and implement automated alerts for future orphaned volumes.

Q3. A practitioner argues that because all S3 data is encrypted with AES-256, the organization is protected against data breaches. Which scenario demonstrates the most direct flaw in this argument?

- A) A. AES-256 can be broken by quantum computers, so the encryption provides only temporary protection.
- B) B. An attacker who obtains valid IAM credentials can call the S3 API and receive plaintext data, because the provider decrypts on behalf of authenticated callers.
- C) C. Server-side encryption does not protect data during transit from S3 to the application.
- D) D. Encryption only protects the most recent version of each object; previous versions remain unencrypted.

Answer: B. An attacker who obtains valid IAM credentials can call the S3 API and receive plaintext data, because the provider decrypts on behalf of authenticated callers.

Q4. What specific technical control addresses the SSRF-based credential theft attack path used in the Capital One breach, and how does it work?

- A) A. Enabling S3 server-side encryption prevents the attacker from reading data even with stolen credentials.
- B) B. Deploying the EC2 instance in a private subnet without internet gateway access prevents all SSRF attacks.
- C) C. Enforcing IMDSv2 requires a session-oriented PUT request to retrieve a token before accessing instance metadata credentials, which most SSRF payloads cannot complete.
- D) D. Using VPC endpoint policies for S3 restricts access to the bucket so that only the VPC's network can reach it.

Answer: C. Enforcing IMDSv2 requires a session-oriented PUT request to retrieve a token before accessing instance metadata credentials, which most SSRF payloads cannot complete.

Q5. A Cloud Security Practitioner is reviewing an organization's storage security controls. Which combination of controls addresses the broadest range of cloud storage threats across all four storage types?

- A) A. AES-256 encryption at rest for all storage types, with TLS for data in transit.
- B) B. Encryption at rest, least-privilege IAM, automated misconfiguration detection, API monitoring, snapshot lifecycle governance, and orphaned resource lifecycle policies.
- C) C. Network segmentation using VPCs and security groups, combined with encryption at rest for all volumes.
- D) D. Regular penetration testing of the cloud environment, combined with encryption of all S3 buckets.

Answer: B. Encryption at rest, least-privilege IAM, automated misconfiguration detection, API monitoring, snapshot lifecycle governance, and orphaned resource lifecycle policies.

Reflection Questions

1. Which storage threats are most directly applicable to your current or target cloud environment? Have you validated that your organization has specific, implemented controls for each one?
2. The Capital One breach began with a misconfigured WAF and ended with over 100 million records exposed through S3. Can you trace the specific failure points and map where a least-privilege IAM policy or IMDSv2 would have changed the outcome?
3. When did your organization last audit for orphaned EBS volumes or other detached storage resources? What process would detect them within 14 days of creation?
4. Has your security program treated encryption as a sufficient control for cloud storage? What specific gaps would you need to close to address the layered defense requirements discussed in this lecture?
5. How would you explain to a non-technical executive why public S3 bucket misconfiguration represents a greater near-term risk than sophisticated adversarial attack techniques?

Key Takeaways

- Misconfiguration — not sophisticated attack techniques — is the most common cause of cloud data exposure, particularly for object storage.
- The Capital One 2019 breach demonstrates how SSRF against an EC2 instance metadata service can yield IAM credentials, and how a least-privilege IAM policy would have contained the impact even after credential compromise.
- Volume storage threats center on unencrypted volumes, exposed snapshots with misconfigured sharing permissions, and orphaned volumes retaining data from decommissioned workloads.
- Ephemeral storage carries real data exposure risk during the instance's active lifetime; the assumption that transience eliminates the need for security controls is a dangerous fallacy.
- Encryption alone is insufficient: it protects the storage medium but does not prevent access by an authenticated API caller with excessive permissions, or by anyone on the internet if a bucket is publicly accessible.
- IMDSv2 on all EC2 instances is a critical control that specifically closes the SSRF-to-credential-theft path used in the Capital One breach.
- A layered cloud storage defense requires encryption, least-privilege IAM, automated misconfiguration detection, API-level monitoring, snapshot lifecycle governance, and orphaned resource management — not any single control in isolation.

Recommended Next Actions

Put this lecture to work within the next seven days:

6. Confirm this week that IMDSv2 is enforced on all EC2 instances in your organization's cloud environment using an AWS Config rule or equivalent compliance check.
7. Run an S3 Access Analyzer scan in your cloud environment and review any externally accessible resources identified; document the business justification for each or revoke the access.

8. Query for all EBS snapshots with sharing permissions broader than the owning account and verify each has a documented justification; revoke any that do not.
9. Review the IAM roles used by your three highest-risk applications accessing object storage: use IAM Access Advisor to identify which permissions have not been used in 90 days and initiate a least-privilege scoping exercise.